

Agent Wiki: A local-first evidence-closed Markdown knowledge base for LLM agents

Zijun Chu^{a,*}

^aIndependent Researcher, Room 1303, No. 19, Lane 899, Xinfeng Middle Road, Qingpu District, Shanghai, China

Abstract

Agent Wiki is an open-source, local-first knowledge substrate for large language model agents. It separates captured evidence from synthesized Markdown wiki pages, resolves ordinary wiki links as a computable graph, and checks an evidence-closure rule before a source is treated as processed. The software also supports source capture, vault search, maintenance queues, link repair, image evidence inventories, optional external-source import, and a read-only graph dashboard. A technical-documentation case study containing 1,137 raw source notes and 127 wiki pages produced 5,444 graph edges with no unresolved links, orphaned wiki pages, or processed-source closure violations under the included lint checks. Agent Wiki provides an inspectable corpus-preparation and maintenance layer that can be used independently or upstream of retrieval-augmented generation.

Keywords: LLM agents, knowledge management, Markdown, local-first software, provenance, software engineering

Required Metadata

Current code version

1. Motivation and significance

Large language model (LLM) agents now perform multi-step reading, coding, and maintenance tasks. These workflows need durable knowledge: source

*Corresponding author.

Email address: `chunimav5@gmail.com` (Zijun Chu)

URL: `https://github.com/NimaChu/agent-wiki` (Zijun Chu)

Nr.	Code metadata description	Current Agent Wiki release
C1	Current code version	Submission candidate based on v0.1.2
C2	Permanent GitHub link to code/repository used for this code version	Final release tag to be inserted before submission; current repository: https://github.com/NimaChu/agent-wiki
C3	Legal Code License	MIT License
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	JavaScript (ECMAScript modules), Node.js, React, Vite, PowerShell
C6	Compilation requirements, operating environments & dependencies	Node.js 18 or newer and npm. The command-line workflows are file-based; the optional dashboard requires a modern web browser. Validated on Windows 11 with Node.js 20.20.0.
C7	Link to developer documentation/manual	https://github.com/NimaChu/agent-wiki/blob/v0.1.2/README.md
C8	Support email for questions	chunimav5@gmail.com

Table 1: Code metadata (mandatory).

material, project conventions, decisions, screenshots, and unresolved follow-ups must remain available after a chat session ends. Existing approaches usually rely on chat history, human-authored note applications, application-specific memory, or retrieval indexes. Each is useful, but independent developers and small research teams still lack a simple way to maintain source-grounded agent knowledge that is private, inspectable, and executable with ordinary developer tools.

Agent Wiki addresses this problem with a local-first repository rather than a hosted database. Local-first software emphasizes user ownership and continued access to data independent of a cloud service [1]. Agent Wiki applies that principle to agent-maintained knowledge: Markdown files are the source of truth, while graph data, dashboards, and future retrieval indexes are derived artifacts. Users can inspect, diff, search, version, or remove the knowledge base without a proprietary export process.

The software is complementary to retrieval-augmented generation (RAG) [2]. RAG optimizes answer-time retrieval, whereas Agent Wiki prepares and audits the corpus before retrieval. It preserves captured evidence, synthesizes reusable knowledge units, and makes maintenance state computable. This is particularly relevant to tool-using and software-engineering agents, which interact with repositories and terminals over extended workflows [3,4].

The intended users are researchers, developers, and technically curious indi-

viduals who want an agent-maintained knowledge base without deploying a database or embedding service. A user clones the repository, installs npm dependencies, and asks a coding agent to operate the included capture, maintenance, search, and validation commands. The same repository rules are readable by both humans and agents.

2. Software description

2.1. Software architecture

Agent Wiki has two persistent knowledge layers and three derived service layers. The raw layer stores source captures, provenance metadata, snapshots, content hashes, image inventories, and processing state. The wiki layer stores atomic, durable pages for concepts, entities, methods, APIs, workflows, comparisons, and recurring questions. Wiki pages link back to raw evidence with Obsidian-compatible links.

The command layer scans both persistent layers and implements capture, search, status, lint, maintenance queues, link repair, image extraction, and optional external-source import. The graph layer resolves links by path, title, basename, and alias, then derives nodes, edges, unresolved targets, relation hints, and incoming/outgoing counts. The optional React/Vite dashboard reads generated JSON and never writes to the vault. Table 2 summarizes the architecture without introducing a runtime database.

Component	Primary representation	Responsibility
Raw evidence	Markdown, snapshots, images	Preserve provenance and source material.
Wiki knowledge	Atomic Markdown pages	Synthesize reusable, source-linked knowledge.
Local CLI	Node.js scripts	Capture, search, maintain, lint, and repair the vault.
Derived graph	In-memory scan and JSON	Resolve links and compute health metrics.
Dashboard	React/Vite web application	Provide read-only graph and evidence inspection.

Table 2: Agent Wiki architecture and ownership boundaries.

2.2. Software functionalities

The central maintenance mechanism is evidence closure. Let r be a raw source, $T(r)$ its declared wiki targets, $R(t)$ the link resolver, and $L(w)$ the links from a wiki page. A processed source must satisfy

$$|T(r)| > 0 \wedge \forall t \in T(r) : R(t) \neq \perp \wedge \exists w \in R(T(r)) : r \in L(w) \wedge \neg \text{followup}(r).$$

The lint command reports missing targets, unresolved targets, missing wiki backlinks, and explicit follow-up flags. It also reports unresolved links, orphaned wiki pages, invalid relation hints, weak pages, and missing metadata. The invariant does not prove semantic correctness; it makes the connection between processed sources and synthesized knowledge auditable.

For visual sources, the image workflow extracts Markdown and HTML image references, resolves relative URLs, mirrors remote images, writes an `image-index.json`, records context and download failures, and inserts a compact inventory into the raw note. Selected screenshots or diagrams can then be promoted into wiki pages while the complete inventory remains in the source layer.

3. Illustrative examples

A minimal installation and health check is:

```
git clone https://github.com/NimaChu/agent-wiki.git
cd agent-wiki
npm install
npm run wiki:status
npm run wiki:lint
```

A user can capture a web source, search the resulting vault, and inspect the optional graph with:

```
npm run wiki:capture -- --title "Source title" \
  --url "https://example.com"
npm run wiki:search -- "query terms"
npm run dashboard:open
```

In normal use, a coding agent reads the repository rules, places captured material in `raw/`, distills reusable concepts into `wiki/`, links claims to evidence, and runs lint before marking a source processed. The dashboard is generated only when requested; it is not required for capture, maintenance, or query workflows.

As an illustrative case study, Agent Wiki maintained a local technical- documentation vault centered on Autodesk FlexSim material. On 2026-07-15, `wiki:status` reported 1,275 nodes, 5,444 edges, 1,137 raw sources, and 127

wiki pages. Of the raw sources, 1,136 were processed and none were pending, stale, or marked for follow-up. `wiki:lint` reported no unresolved links, invalid relations, processed-source issues, orphaned or weak wiki pages, missing claim sources, or required metadata omissions. This result measures mechanical graph health, not factual answer quality.

4. Impact

Agent Wiki changes the unit of agent memory from an opaque application state to a reviewable software artifact. This has three practical effects. First, provenance survives synthesis: a concept page can be traced to one or more raw notes and locally mirrored visual evidence. Second, maintenance becomes testable: a user or continuous-integration job can run the same status and lint commands as the agent. Third, the knowledge base remains reusable: Markdown can feed text search, graph navigation, embeddings, RAG, or future Model Context Protocol services without changing the source-of-truth format.

The case study provides initial evidence that the design can maintain link and closure health across more than one thousand source notes. Its value is not the node count alone but the absence of unresolved links, orphaned wiki pages, and processed-source violations under executable checks. These properties support research workflows in which literature, documentation, protocols, or software knowledge must remain inspectable while agents update the synthesis layer.

The software is domain independent. Its repository conventions can be applied to technical documentation, literature reviews, laboratory procedures, policy collections, or project knowledge. The raw/wiki separation also supports a privacy-preserving publication model: reusable tooling can be shared publicly while private or bulky knowledge remains outside version control.

Current adoption evidence is limited to a single-developer, single-corpus case study. Agent Wiki has not yet been evaluated through a controlled user study or against chat-only and RAG-first baselines. Future work should measure answer quality, maintenance effort, cross-agent consistency, and user trust; publish a sanitized case-study snapshot; and evaluate optional claim-level citations and embedding indexes. The public software distribution and archived release [5] allow these questions to be pursued without depending on a hosted service.

5. Conclusions

Agent Wiki provides a local-first, Markdown-native substrate for durable LLM- agent knowledge. Its principal software contribution is an executable

workflow that separates raw evidence from synthesized wiki pages and checks evidence closure before sources are treated as processed. Capture, search, lint, image evidence, graph generation, and dashboard inspection are available through a small set of local commands. The documentation case study demonstrates mechanical health at more than one thousand raw sources while also defining the limits of the present evidence. Agent Wiki is intended as an inspectable layer upstream of, or independent from, retrieval systems.

Current executable software version

Nr.	Executable software meta-data description	Current Agent Wiki release
S1	Current software version	Submission candidate based on v0.1.2
S2	Permanent link to executables of this version	Final release link to be inserted before submission.
S3	Permanent link to reproducible capsule/archive	https://doi.org/10.5281/zenodo.21252490
S4	Legal Software License	MIT License
S5	Computing platforms/operating systems	Node.js 18+ environment; validated on Windows 11.
S6	Installation requirements and dependencies	Node.js 18+ and npm; run <code>npm install</code> .
S7	Link to user manual	https://github.com/NimaChu/agent-wiki/blob/v0.1.2/README.md
S8	Support email for questions	chunimav5@gmail.com

Table 3: Executable software metadata (optional).

Data availability

The reusable software and manuscript materials are available from GitHub and Zenodo [5]. The reported case-study vault includes locally captured technical documentation and is not part of the public software release. The aggregate health results can be regenerated from that vault with `npm run wiki:status` and `npm run wiki:lint`. A sanitized replication snapshot is planned for a future release.

CRedit author statement

Zijun Chu: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Writing—original draft, Writing—review and editing, Visualization, Project administration.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author used OpenAI Codex to assist with manuscript organization, language drafting, and LaTeX preparation. The author reviewed and edited the content as needed and takes full responsibility for the content of the article.

Acknowledgements

The author thanks the open-source communities whose tools and documentation support local-first software development.

References

- [1] Kleppmann M, Wiggins A, van Hardenberg P, McGranaghan M. Local-first software: You own your data, in spite of the cloud. In: Proc ACM SIGPLAN Int Symp New Ideas, New Paradigms, Reflections Program Softw; 2019, p. 154–178. <https://doi.org/10.1145/3359591.3359737>.
- [2] Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. Adv Neural Inf Process Syst 2020;33:9459–74. <https://arxiv.org/abs/2005.11401>.
- [3] Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, et al. ReAct: Synergizing reasoning and acting in language models. In: Int Conf Learn Represent; 2023. <https://arxiv.org/abs/2210.03629>.
- [4] Yang J, Jimenez CE, Wettig A, Lieret K, Yao S, Narasimhan K, et al. SWE-agent: Agent-computer interfaces enable automated software engineering. Adv Neural Inf Process Syst 2024;37. <https://arxiv.org/abs/2405.15793>.

- [5] Chu Z. Agent Wiki: A local-first evidence-closed Markdown wiki for LLM agents. Version 0.1.2 [software]. Zenodo; 2026 Jul 8. <https://doi.org/10.5281/zenodo.21252490>.